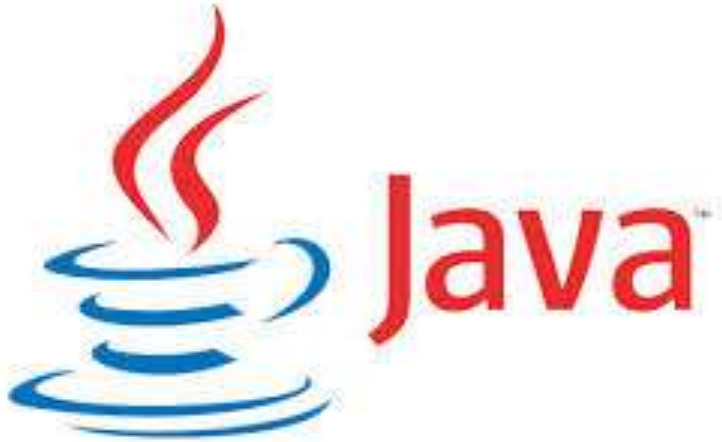


SOC2030 Application Programming in Java

Spring framework

Dr. Andrei Dragunov

Why Use Any Framework?

A **general purpose programming language like Java is capable of supporting a wide variety of applications**. Not to mention that Java is actively being worked upon and improving every day.

So why do we need a framework after all? Honestly, it isn't absolutely necessary to use a framework to accomplish a task. But, it's often advisable to use one for several reasons:

- Helps us **focus on the core task rather than the boilerplate** associated with it
- Brings together years of wisdom in the form of design patterns
- Helps us adhere to the industry and regulatory standards
- Brings down the total cost of ownership for the application

Spring Framework

Spring framework is divided into modules which makes it really easy to pick and choose in parts to use in any application:

- **Core**: Provides core features like DI (Dependency Injection), Internationalisation, Validation, and AOP (Aspect Oriented Programming)
- **Data Access**: Supports data access through JTA (Java Transaction API), JPA (Java Persistence API), and JDBC (Java Database Connectivity)
- **Web**: Supports both Servlet API (**Spring MVC**) and of recently Reactive API (**Spring WebFlux**), and additionally supports WebSockets, STOMP, and WebClient
- **Integration**: Supports integration to Enterprise Java through JMS (Java Message Service), JMX (Java Management Extension), and RMI (Remote Method Invocation)
- **Testing**: Wide support for unit and integration testing through Mock Objects, Test Fixtures, Context Management, and Caching

Spring Projects

What makes Spring much more valuable is **a strong ecosystem that has grown around it over the years and that continues to evolve actively**. These are structured as Spring projects which are developed on top of the Spring framework.

- **Boot**: Provides us with a set of highly opinionated but extensible template for creating various projects based on Spring in almost no time. It makes it really easy to create standalone Spring applications with embedded Tomcat or a similar container.
- **Cloud**: Provides support to easily develop some of the common distributed system patterns like service discovery, circuit breaker, and API gateway. It helps us cut down the effort to deploy such boilerplate patterns in local, remote or even managed platforms.
- **Security**: Provides a robust mechanism to develop authentication and authorization for projects based on Spring in a highly customizable manner. With minimal declarative support, we get protection against common attacks like session fixation, click-jacking, and cross-site request forgery.
- **Mobile**: Provides capabilities to detect the device and adapt the application behavior accordingly. Additionally, supports device-aware view management for optimal user experience, site preference management, and site switcher.
- **Batch**: Provides a lightweight framework for developing batch applications for enterprise systems like data archival. Has intuitive support for scheduling, restart, skipping, collecting metrics, and logging. Additionally, supports scaling up for high-volume jobs through optimization and partitioning.

Advantages of Spring Framework

1) Predefined Templates

Spring framework provides templates for JDBC, Hibernate, JPA etc. technologies. So there is no need to write too much code. It hides the basic steps of these technologies. Let's take the example of JdbcTemplate, you don't need to write the code for exception handling, creating connection, creating statement, committing transaction, closing connection etc. You need to write the code of executing query only. Thus, it save a lot of JDBC code.

2) Loose Coupling

The Spring applications are loosely coupled because of dependency injection.

3) Easy to test

The Dependency Injection makes easier to test the application. The EJB or Struts application require server to run the application but Spring framework doesn't require server.

4) Lightweight

Spring framework is lightweight because of its POJO implementation. The Spring Framework doesn't force the programmer to inherit any class or implement any interface. That is why it is said non-invasive.

5) Fast Development

The Dependency Injection feature of Spring Framework and it support to various frameworks makes the easy development of JavaEE application.

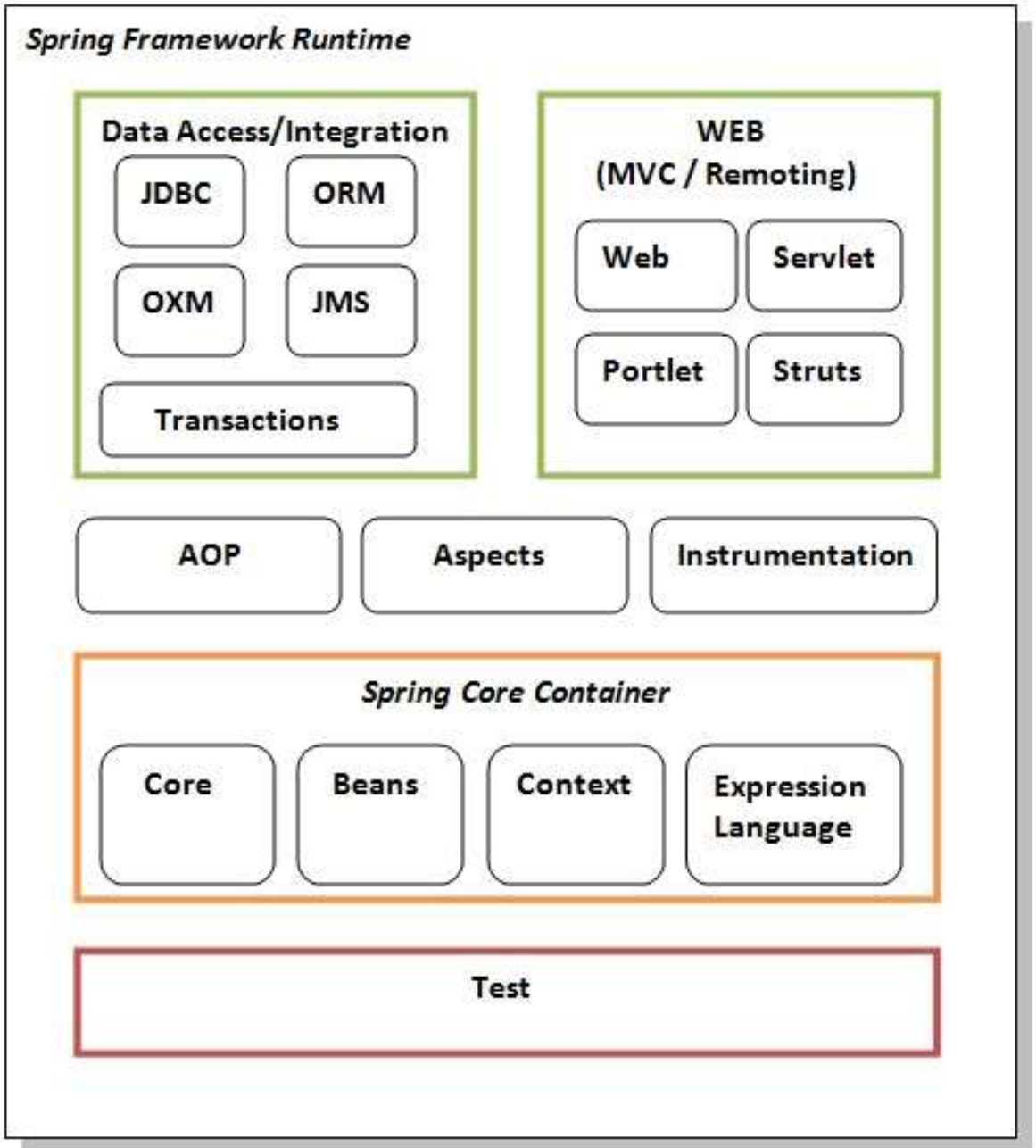
6) Powerful abstraction

It provides powerful abstraction to JavaEE specifications such as JMS, JDBC, JPA and JTA.

7) Declarative support

It provides declarative support for caching, validation, transactions and formatting.

Spring Modules



Benefits of Using the Spring Framework

- Spring enables developers to develop enterprise-class applications using POJOs. The benefit of using only POJOs is that you do not need an EJB container product such as an application server but you have the option of using only a robust servlet container such as Tomcat or some commercial product.
- Spring is organized in a modular fashion. Even though the number of packages and classes are substantial, you have to worry only about the ones you need and ignore the rest.
- Spring does not reinvent the wheel, instead it truly makes use of some of the existing technologies like several ORM frameworks, logging frameworks, JEE, Quartz and JDK timers, and other view technologies.
- Testing an application written with Spring is simple because environment-dependent code is moved into this framework. Furthermore, by using JavaBeanstyle POJOs, it becomes easier to use dependency injection for injecting test data.
- Spring's web framework is a well-designed web MVC framework, which provides a great alternative to web frameworks such as Struts or other over-engineered or less popular web frameworks.
- Spring provides a convenient API to translate technology-specific exceptions (thrown by JDBC, Hibernate, or JDO, for example) into consistent, unchecked exceptions.
- Lightweight IoC containers tend to be lightweight, especially when compared to EJB containers, for example. This is beneficial for developing and deploying applications on computers with limited memory and CPU resources.
- Spring provides a consistent transaction management interface that can scale down to a local transaction (using a single database, for example) and scale up to global transactions (using JTA, for example).

What Is Inversion of Control?

Inversion of Control is a principle in software engineering which transfers the control of objects or portions of a program to a container or framework. We most often use it in the context of object-oriented programming.

In contrast with traditional programming, in which our custom code makes calls to a library, IoC enables a framework to take control of the flow of a program and make calls to our custom code. To enable this, frameworks use abstractions with additional behavior built in. **If we want to add our own behavior, we need to extend the classes of the framework or plugin our own classes.**

The advantages of this architecture are:

- decoupling the execution of a task from its implementation
- making it easier to switch between different implementations
- greater modularity of a program
- greater ease in testing a program by isolating a component or mocking its dependencies, and allowing components to communicate through contracts

Dependency Injection (DI)

The technology that Spring is most identified with is the **Dependency Injection (DI)** flavor of Inversion of Control.

The **Inversion of Control (IoC)** is a general concept, and it can be expressed in many different ways. Dependency Injection is merely one concrete example of Inversion of Control.

When writing a complex Java application, application classes should be as independent as possible of other Java classes to increase the possibility to reuse these classes and to test them independently of other classes while unit testing. Dependency Injection helps in gluing these classes together and at the same time keeping them independent.

What is dependency injection?

- What is dependency injection exactly? Let's look at these two words separately. Here the dependency part translates into an association between two classes. For example, class A is dependent of class B. Now, let's look at the second part, injection. All this means is, class B will get injected into class A by the IoC.
- Dependency injection can happen in the way of passing parameters to the constructor or by post-construction using setter methods.

Aspect Oriented Programming (AOP)

- One of the key components of Spring is the **Aspect Oriented Programming (AOP)** framework. The functions that span multiple points of an application are called **cross-cutting concerns** and these cross-cutting concerns are conceptually separate from the application's business logic. There are various common good examples of aspects including logging, declarative transactions, security, caching, etc.
- The key unit of modularity in OOP is the class, whereas in AOP the unit of modularity is the aspect. DI helps you decouple your application objects from each other, while AOP helps you decouple cross-cutting concerns from the objects that they affect.
- The AOP module of Spring Framework provides an aspect-oriented programming implementation allowing you to define method-interceptors and pointcuts to cleanly decouple code that implements functionality that should be separated. We will discuss more about Spring AOP concepts in a separate chapter.

Spring - Bean Definition

The objects that form the backbone of your application and that are managed by the Spring IoC container are called **beans**. A bean is an object that is instantiated, assembled, and otherwise managed by a Spring IoC container. These beans are created with the configuration metadata that you supply to the container. For example, in the form of XML <bean/> definitions which you have already seen in the previous chapters.

Bean definition contains the information called **configuration metadata**, which is needed for the container to know the following −

- How to create a bean?
- Bean's lifecycle details?
- Bean's dependencies?

Spring - Dependency Injection

- Every Java-based application has a few objects that work together to present what the end-user sees as a working application. When writing a complex Java application, application classes should be as independent as possible of other Java classes to increase the possibility to reuse these classes and to test them independently of other classes while unit testing. Dependency Injection (or sometime called wiring) helps in gluing these classes together and at the same time keeping them independent.

- Example in netBeans

```
public abstract class Paper {
    public abstract void prnType();
}
@Component("white")
//@Primary
public class WhitePaper extends Paper{

    @Override
    public void prnType() {
        System.out.println("U use white
paper");
    }

}
```

```
@Component
public class TestPrinter {
    @Autowired
    @Qualifier("white")
    private Paper p;

    public void setP(Paper p) {
        this.p = p;
    }

    public Paper getP() {
        return p;
    }

    public void print(){
        System.out.println("Hello Spring");
        p.prnType();
    }
}
```

```
public class SpringTest {


    public static void main(String[] args) {
        ApplicationContext context = new AnnotationConfigApplicationContext(AppConfig.class);
        //Paper p = context.getBean(WhitePaper.class);
        TestPrinter tp = context.getBean(TestPrinter.class);
        //tp.setP(p);
        tp.print();

    }

}
```

DI exists in two major variants:

Sr.No.	Dependency Injection Type & Description
1	**Constructor-based dependency injection** Constructor-based DI is accomplished when the container invokes a class constructor with a number of arguments, each representing a dependency on the other class.
2	**Setter-based dependency injection** Setter-based DI is accomplished by the container calling setter methods on your beans after invoking a no-argument constructor or no-argument static factory method to instantiate your bean.

Sp

- Fc
co
at
de

Sr.No	Mode & Description
1	**no** This is default setting which means no autowiring and you should use explicit bean reference for wiring. You have nothing to do special for this wiring. This is what you already have seen in Dependency Injection chapter.
2	**byName** Autowiring by property name. Spring container looks at the properties of the beans on which autowire attribute is set to byName in the XML configuration file. It then tries to match and wire its properties with the beans defined by the same names in the configuration file.
3	**byType** Autowiring by property datatype. Spring container looks at the properties of the beans on which autowire attribute is set to byType in the XML configuration file. It then tries to match and wire a property if its **type** matches with exactly one of the beans name in configuration file. If more than one such beans exists, a fatal exception is thrown.

n

Cont…

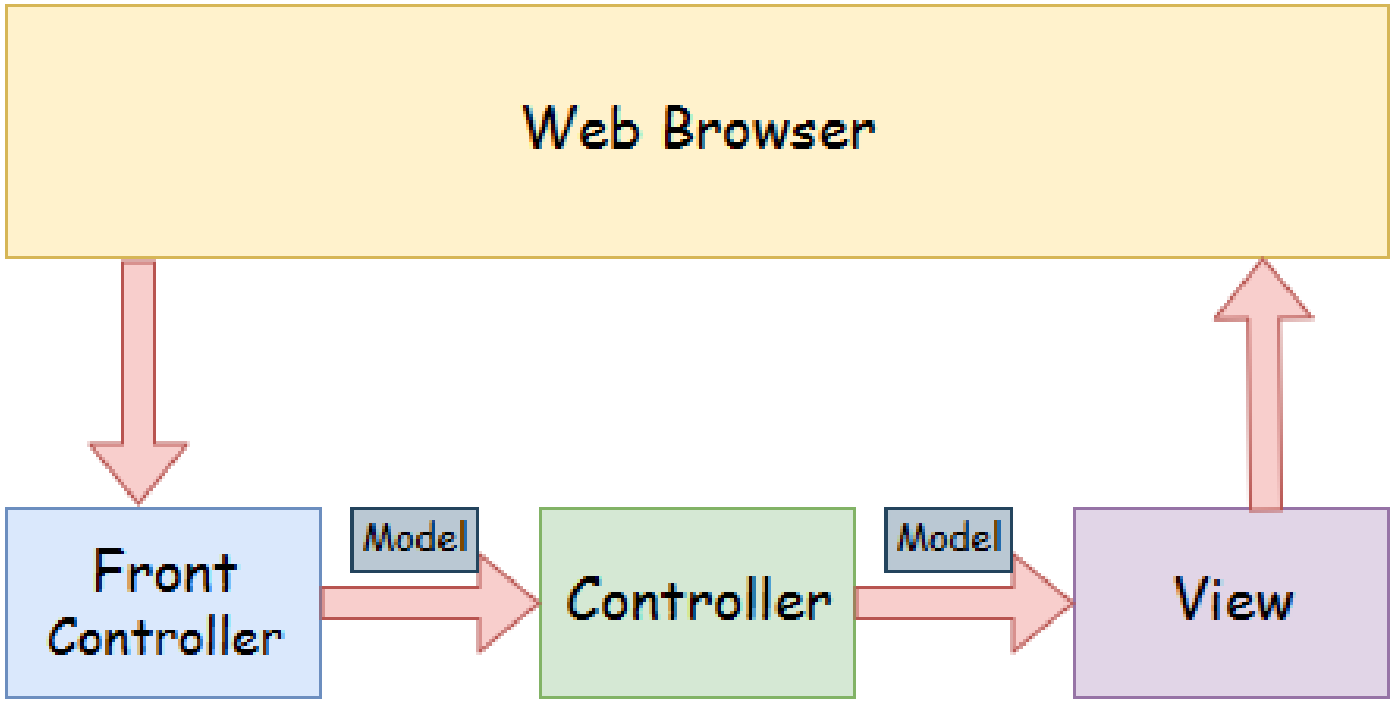
4	**constructor** Similar to byType, but type applies to constructor arguments. If there is not exactly one bean of the constructor argument type in the container, a fatal error is raised.
5	**autodetect** Spring first tries to wire using autowire by constructor, if it does not work, Spring tries to autowire by byType.

Spring MVC Hello World Example

Steps	Description
1	Create a Dynamic Web Project with a name HelloWeb and create a package com.tutorialspoint under the src folder in the created project.
2	Drag and drop below mentioned Spring and other libraries into the folder WebContent/WEB-INF/lib.
3	Create a Java class HelloController under the com.tutorialspoint package.
4	Create Spring configuration files web.xml and HelloWeb-servlet.xml under the WebContent/WEB-INF folder.
5	Create a sub-folder with a name jsp under the WebContent/WEB-INF folder. Create a view file hello.jsp under this sub-folder.
6	The final step is to create the content of all the source and configuration files and export the application as explained below.

Spring Web Model-View-Controller

- A Spring MVC provides an elegant solution to use MVC in spring framework by the help of **DispatcherServlet**. Here, **DispatcherServlet** is a class that receives the incoming request and maps it to the right resource such as controllers, models, and views.



- **Model** - A model contains the data of the application. A data can be a single object or a collection of objects.
- **Controller** - A controller contains the business logic of an application. Here, the @Controller annotation is used to mark the class as the controller.
- **View** - A view represents the provided information in a particular format. Generally, JSP+JSTL is used to create a view page. Although spring also supports other view technologies such as Apache Velocity, Thymeleaf and FreeMarker.
- **Front Controller** - In Spring Web MVC, the DispatcherServlet class works as the front controller. It is responsible to manage the flow of the Spring MVC application.

Understanding the flow of Spring Web MVC

- As displayed in the figure, all the incoming request is intercepted by the DispatcherServlet that works as the front controller.
- The DispatcherServlet gets an entry of handler mapping from the XML file and forwards the request to the controller.
- The controller returns an object of ModelAndView.
- The DispatcherServlet checks the entry of view resolver in the XML file and invokes the specified view component.